

PDF Tagger SNS

Tech stack and how tagging works · client-side only · no server, no ML model

This app is a browser-only Accessible Ebook Tagger. A PDF never leaves the machine. Tags are inferred from layout heuristics (font size, position, regex), then you can correct them by hand. Export writes a simplified tagged-PDF catalog with pdf-lib.

Tech stack

Layer	What it uses
UI	React 19 + TypeScript
Build	Vite 8
Styling	Tailwind CSS 4, Lucide icons
Read / render PDF	pdfjs-dist (text items, page canvas, worker)
Write / export	pdf-lib (Lang, MarkInfo, StructTreeRoot)
Checks / speech	Custom TS: accessibilityValidator, speechSynthesizer
Alt text	Rule templates in altTextGenerator.ts — not an API

How tagging happens

- 1. Parse.** `parsePdfFile` in `src/services/pdfParser.ts` reads the file, opens it with PDF.js, pulls metadata, then for each page takes `getTextContent()` (string + transform + size). Pages with no text get a placeholder paragraph flagged for review — OCR is simulated, not real.
- 2. Group.** Glyphs cluster by baseline Y, then merge into blocks when the vertical gap or font-size jump is large. Each block gets a bounding box as 0–100% of the page.
- 3. Classify.** Default tag is P. Then these rules fire:

Signal	Tag
Near top (y < 6) and small font	Artifact (header)
Near bottom (y > 93) and very short	Artifact (footer / page no.)
Font ≥ 20, or ≥ 18 on page 1	H1
Font ≥ 14	H2
Font ≥ 12, short, numbered like 1.2 Title	H3
Starts with Figure / Fig. / Illustration	Caption
Starts with 1. * - •	ListItem
Looks like a URL	Link

- 4. Reading order.** Sort by page, then header artifacts, then left vs right column (x > 45 and narrow width), then top to bottom. Renumber 1...n.
- 5. Human correction.** Click a box or the structure tree. A manual tag sets confidence to 1.0. Figures can get template alt text from nearby heading/caption and keyword rules.
- 6. Check and export.** `evaluateAccessibility` scores heading, alt-text, order, table, and language rules. pdf-lib writes /Lang, Marked=true, and a StructTreeRoot of StructElem nodes. Artifacts are omitted. The tree is a simplified overlay (not full marked-content wiring). JSON and HTML-for-EPUB are also available.
- Bottom line.** PDF.js extracts positioned text → heuristics tag the blocks → you retag by hand → pdf-lib writes a tagged-PDF-ish catalog. No neural classifier. No server.